

# TEORÍA DE ALGORITMOS 1

## Clases de complejidad - Reducciones avanzadas

Por: ING. VÍCTOR DANIEL PODBEREZSKI  
vpodberezski@fi.uba.ar

---

### 1. Coloreo de grafos

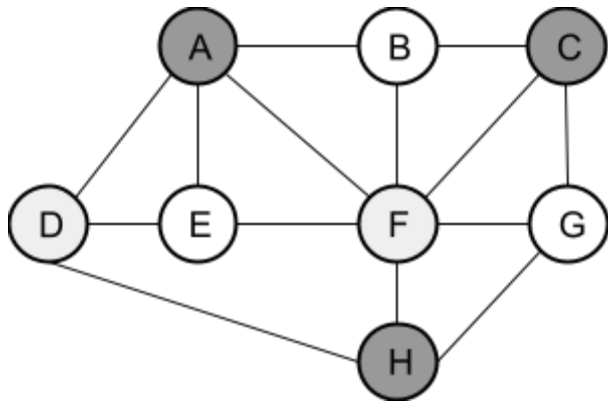
El siguiente problema corresponde a uno de etiquetado de nodos. En este tipo de problemas se busca asignar a cada nodo del grafo un valor identificador que cumpla con ciertos requisitos. En el caso de coloreo de grafos, estos valores o etiquetas representan colores. En su versión de optimización se busca asignar la menor cantidad de colores a los nodos del grafo. Existe una restricción: dos nodos adyacentes no pueden compartir el mismo color. En su versión de decisión se pregunta si es posible encontrar una función de coloreo sin superar un número dado de colores.

El interés en este problema surge históricamente en cartografía: la posibilidad de pintar regiones políticas de un mapa evitando que limítrofes utilicen el mismo color. El mapa puede representarse como un grafo donde los nodos son las regiones y los ejes unen a aquellas regiones con un límite en común. En esta versión del problema el grafo cumple con ciertas características especiales. Así como este, fueron apareciendo otros problemas de coloreo de interés, entre ellos por ejemplo para grafos planares o donde el objeto a colorear es el eje.

El coloreo de grafos tiene usos prácticos que lo hacen un problema de estudio de sumo intereses. Lograr un coloreo implica separar los nodos del grafos en sets donde cada elemento es independiente del resto. Es decir no están vinculados entre sí por un eje. Un caso de aplicación sería asignar frecuencias limitadas a antenas de comunicación. Logrando que aquellas que comparten la misma no pueden interferirse entre sí. También se puede utilizar en tareas de programación con un número restringido de recursos. Se lo estudia además para desarrollo de circuitos electrónicos y otras situaciones.

### Coloreo de grafos (CG)

Dado un grafo  $G=(V,E)$  no dirigido, y un número  $k$  entero. Es posible definir una función  $f$  de coloreo tal que asigne un color  $0 < c_i \leq k$  a cada nodo  $i$  del grafo, tal que no existan nodos adyacentes con el mismo color?



En el grafo de la imagen se muestra una función de coloreo de  $k=3$ . Se puede observar que no existen dos nodos unidos por un eje que compartan el mismo color. Por lo tanto corresponde a un 3-coloreo válido. Si necesitamos que sea un 4 coloreo, podemos tomar por ejemplo el nodo "A" y asignarle un color diferente. También se puede observar que es imposible realizar una 2-coloración en este

grafo. Si tomamos los nodos A,B y F observamos que cualquier intento de modificar el color de uno de ellos por otro existente lo hará imposible.

Comenzamos definiendo un certificado y certificador polinomial que para cualquier instancia del problema nos permita afirmar o no que existe para una instancia determinada un conjunto independiente de tamaño  $k$ . La evidencia será en este caso, una asignación de colores a cada uno de los nodos del grafo. Es decir que la evidencia es la función de coloración que alguien nos presenta. El certificador deberá recibir por parámetro la instancia del problema (El grafo y el valor de  $k$ ) y la evidencia (una posible asignación de colores a los nodos. Deberá probar en tiempo polinomial que:

- Todos los nodos del grafo aparecen en la evidencia una y solo una vez
- Cada nodo recibe un valor de coloración entre 1 y  $k$ .
- Todos los colores son utilizados
- Ningún par de nodos adyacentes tiene el mismo color.

---

La complejidad del conjunto de estas tareas se puede realizar en  $O(V+E)$ . Por lo tanto podemos afirmar que corresponde a un problema en NP.

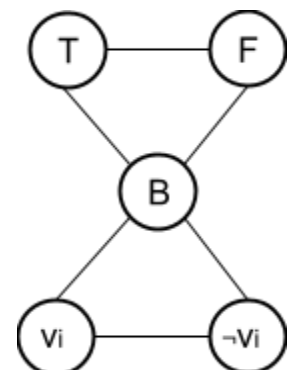
Para probar que el problema pertenece a NP-H vamos a utilizar 3-SAT. Concretamente realizaremos la reducción  $3\text{-SAT} \leq_p 3\text{-CG}$ . Es decir que intentaremos poder resolver cualquier instancia de 3-SAT mediante un 3 coloreo de grafos. Podemos de forma sencilla para  $3 < k < |V|$  realizar una reducción polinomial de 3-CG a K-CG. Si en un grafo podemos armar 3 set de nodos que sean independientes entre sí (no tienen ejes que los unan), entonces podemos seleccionar k-3 nodos de estos y asignarles respectivamente el color 4,5,...k. El único cuidado que debemos tener es dejar al menos 1 nodo en los 3 sets iniciales. ¿Qué pasa con el 2-coloreo de grafos? Se puede demostrar que para  $k=2$  el problema pertenece a P.

La reducción no será tan sencilla como las evaluadas anteriormente. Utilizaremos para ayudarnos "gadgets" (artilugios). Un "gadget" es un pequeño dispositivo especializado para alguna tarea particular. Como en la reducción trataremos de crear una instancia de 3-SAT en un grafo. Nuestros artilugios serán subgrafos que utilizaremos para lograr nuestro cometido.

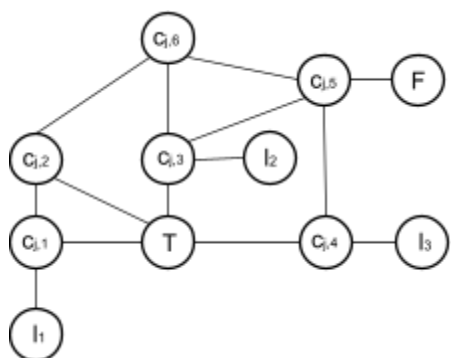
Partimos de una expresión booleana  $\phi$  que deseamos saber si se puede satisfacer. Esta tiene "n" variables y "c" cláusulas. Cada cláusula tiene exactamente 3 literales. Queremos construir un grafo  $G=(V,E)$  y utilizando el parámetro " $K=3$ " basados en  $\phi$ , tal que, cuando sea 3-coloreable implique que  $\phi$  se puede satisfacer.

Pensaremos los colores como posibles estados que pueden tomar las variables. Serán los colores "True", "False" y el tercero corresponderá a un estado "base" (o neutro). Crearemos un nodo por cada color. Los llamaremos "T", "F" y "B". Estos nos ayudarán luego a restringir como colorear el grafo.

Cada variable  $v_i$  estará representada por dos nodos. Un nodo será la variable negada y la otra sin negar. Buscaremos que no puedan tener el mismo color en su coloreo. Por eso en nuestro grafo esos nodos estarán unidos por un eje. Además los uniremos con los nodos de color conformando el gadget "variable". El mismo se muestra en la figura. Como se puede observar  $v_i$  y  $\neg v_i$  sólo podrán ser pintados



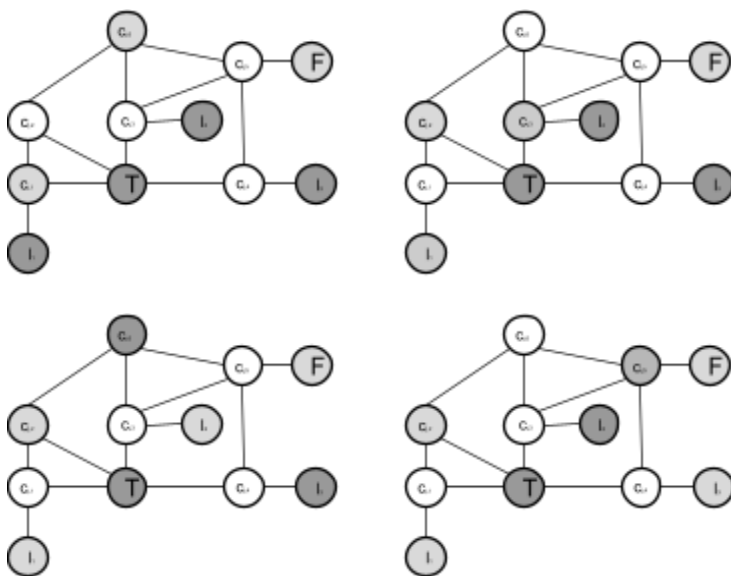
alternativamente con los mismos colores del nodo "T" y "F". Si tenemos  $n$  variables entonces tendremos  $2n$  nodos en pares y cada uno unido con el nodo base.

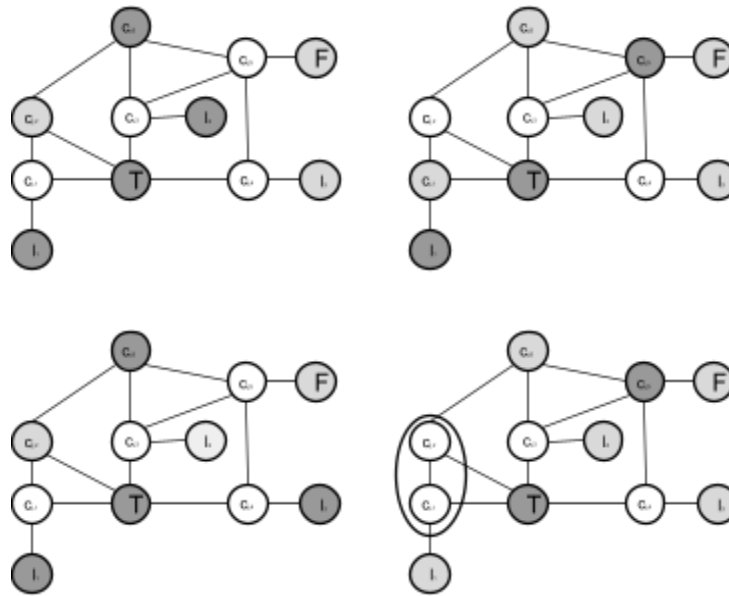


Debemos poder representar ahora las cláusulas y sus restricciones de colores. Cada cláusula  $j$  está conformada por 3 literales que son las variables del problema con o sin negar. Estas ya las representamos anteriormente como nodos. Debemos vincular estas entre sí para lograr que al menos una de ellas tenga el valor (color) necesario para representar la satisfacción de la cláusula. Crearemos un gadget "cláusula" por cada una de ellas.

Esta requiere 6 nodos " $C_j$ " que vinculan los literales con los nodos de los colores "T" y "F". Se disponen de tal forma para lograr que la única forma de 3-colorear el grafo total se logra si alguno los 3 literales del gadget toma el color "T" (True). Los nodos literales no hay que crearlos, son los nodos variables (el negado o el sin negar).

En la siguiente imagen se muestran las diferentes formas de colorear el gadget y la imposibilidad de asignar el valor "F" a todos los literales del mismo. Si todos los literales se intentan asignar el color "F" (que representa en nuestro problema de 3-SAT al "false") tendremos indefectiblemente nodos adyacentes que necesitan un cuarto color.

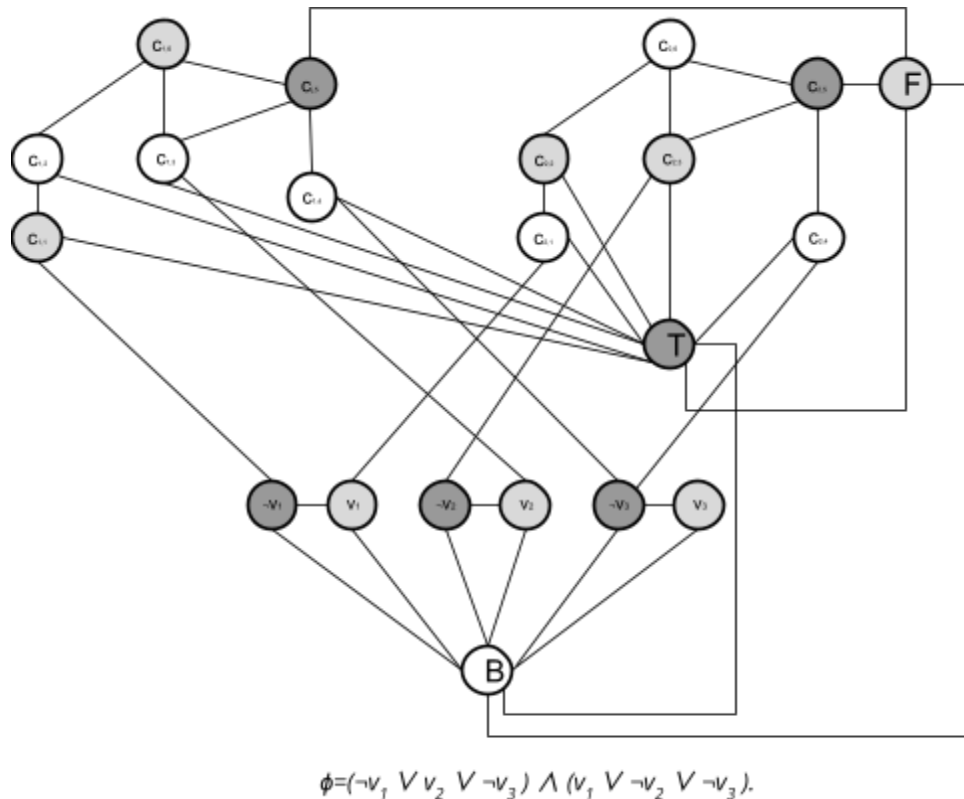




La reducción entonces construye el grafo con 3 nodos para los colores,  $2 \cdot n$  nodos para las “n” variables de la fórmula y 6 nodos por cada cláusula. Además se crean 3 ejes para los nodos colores, 3 ejes por cada variable y 14 ejes por cada cláusula. El grafo resultante en definitiva se crea con un número polinomial de operaciones. Utilizamos este grafo y  $k=3$  y lo enviamos a nuestra caja negra que resuelve el problema del k-coloreo. La respuesta será “si (o no) se puede colorear el grafo”. Y resultará la misma respuesta por el sí o por el no para la satisfacción del problema de 3-sat.

Concluimos que por su pertenencia a NP y NP-Hard el coloreo de grafos corresponde a un problema NP-C.

*Supongamos que queremos saber si se puede satisfacer la siguiente función  $\phi = (\neg v_1 \vee v_2 \vee \neg v_3) \wedge (v_1 \vee \neg v_2 \vee \neg v_3)$ . Tenemos 2 cláusulas y 3 variables. Construiremos entonces 2 gadget variables y 3 gadget cláusulas. En la siguiente imagen se muestra la reducción y una posible coloración para la satisfacción.*



La interpretación de la coloración nos brinda una posible asignación de valor de las variables para satisfacer cada una de las cláusulas y consecuentemente satisfacer la función.  $v_1 = \text{False}$ ,  $v_2 = \text{false}$  y  $v_3 = \text{false}$ . Podrían encontrarse otras coloraciones válidas y por lo tanto otras asignaciones de variables que también satisfagan. Pero la existencia de al menos una, es decir la posibilidad de colorear, nos permite poder afirmar que esta instancia de 3-SAT se puede satisfacer.

## 2. 3-dimensional matching

El siguiente problema es una variante del emparejamientos en conjuntos disjuntos. En ese caso teníamos 2 set de elementos que admitía la formación de ciertas parejas. El emparejamiento o matching de dos elementos impide que estos puedan formar pareja con otros con los que podrían hacerlo. Pudimos resolver este problema en una reducción a un problema de flujo máximo. Utilizando edmonds-Karp u otro algoritmo polinomial podemos resolver el problema en tiempo polinomial.

---

En esta variante aumentamos en uno la cantidad de conjuntos. Una pareja estará compuesta por una tripla donde tendremos un elemento de cada conjunto. Tenemos como dato cuales son las triplas compatibles y candidatas a ser seleccionadas en el conjunto final. En su versión de optimización buscaremos construir la mayor cantidad de triplas. En su versión de decisión nos preguntaremos si es posible seleccionar triplas de tal forma que ningún elemento de ningún conjunto se quede sin emparejar: un matching perfecto.

|                                     |
|-------------------------------------|
| <b>3-dimensional matching (3DM)</b> |
|-------------------------------------|

|                                                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dados 3 sets disjuntos $X, Y, Z$ de tamaño $n$ cada uno, y un set $C$ conformado por triplas $(x, y, z)$ de triplas con $x \in X, y \in Y, z \in Z$ . Es posible seleccionar un subset de $n$ triplas en $C$ tal que cada elemento de $X, Y, Z$ sea contenido exactamente en una de esas triplas? |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Contamos con 3 conjuntos: instrumentistas, cirujanos y anestesiólogos. Cada uno de ellos con 5 profesionales. Queremos armar 5 triplas para la próxima jornada de quirófano. Lamentablemente no todos pueden trabajar bien entre sí. Por lo que se ha realizado un detalle de triplas compatibles. Los instrumentistas son {Ana, Bruno, Clara, Diego, Elvira}, los cirujanos {Florencia, Gastón, Horacio, Ifigenia, Jorge} y los anestesiólogos {Kevin, Luisa, Martín, Natalio, Omar}. Los siguientes son los equipos que se pueden armar {Ana, Gastón, Kevin}, {Ana, Jorge, Luisa}, {Ana, Jorge, Omar}, {Bruno, Ifigenia, Kevin}, {Clara, Gastón, Martín}, {Clara, Jorge, Martín}, {Diego, Horacio, Kevin}, {Diego, Horacio, Luisa}, {Elvira, Florencia, Natalio}. Si seleccionamos a {Ana, Gastón, Kevin}, nos imposibilitaba seleccionar las triplas que tengan a alguno de ellos. Por ejemplo no podre seleccionar a {Bruno, Ifigenia, Kevin}. Pero al ser esta última la única en la que está Bruno, entonces no podremos cumplir con la formación de 5 parejas. Si seleccionamos a {Bruno, Ifigenia, Kevin}, {Elvira, Florencia, Natalio}, {Ana, Jorge, Omar}, {Clara, Gastón, Martín}, {Diego, Horacio, Luisa} logramos que todos estén en conjuntos, sin repetirse. Por lo tanto, en nuestro ejemplo, es posible lograr armar los 5 equipos necesarios.

Ya presentado el problema, demostraremos que el mismo pertenece a la clase NP-C. En primer lugar un certificador polinomial nos ayudará a clasificarlo como perteneciente a NP. En este caso nuestra evidencia será un set con las posibles triplas de  $C$  para armar la solución. Con esto debemos:

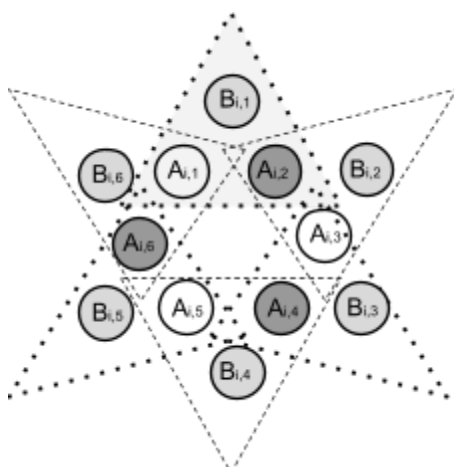
- Verificar que las triplas en la evidencia son “n”
- Verificar que cada elemento de la tripla corresponde al conjunto correspondiente X,Y,Z.
- Verificar que ningún elemento falta o se repite
- Verificar que cada tripla de la evidencia es una tripla disponible en C.

Cada uno de estos pasos puede ser analizado en tiempo polinomial en función de “n” y la cantidad máxima de triplas candidatas en C (que si no hay repeticiones también se puede acotar por “n”). Este certificador es polinomial y por lo tanto el problema pertenece a NP.

Para demostrar que el problema pertenece a NP-H realizaremos la reducción  $3\text{-SAT} \leq_p 3\text{DM}$ . De esa forma si cualquier instancia de 3SAT se puede resolver con una instancia de 3DM, podremos afirmar que 3DM es un problema igual o más difícil que todos los demás en NP. La transformación entre una instancia de 3-SAT y una de 3DM no es sencilla. Iremos paso a paso. El objetivo es pasar de una expresión booleana de “n” variables y “k” cláusulas a 3 conjuntos y un set de triplas. Si en esa instancia creada se pueden seleccionar las triplas para cubrir todos los elementos de los conjuntos cumpliendo con las restricciones de seleccionar 1 y solo 1 vez cada elementos, entonces la expresión de 3-Sat se puede satisfacer.

Comenzamos con los sets X,Y y Z vacíos. De igual manera el conjunto C de triplas no tendrá elementos inicialmente.

En primer lugar crearemos un gadget “variable” por cada variable  $v_i$  de la instancia de 3sat. En cada gadget agregaremos 4 elementos por cada cláusula de la función booleana. 2 de



cada uno de esos elementos los llamaremos “puntas” y las otras 2 “núcleo”. Estos nombres cobran sentido con la representación visual en forma de estrellas que se le da al gadget a la hora de explicar su elaboración. En la imagen que acompaña al párrafo se puede ver un gadget variable genérico para una variable en una expresión booleana que tiene 3 cláusulas.

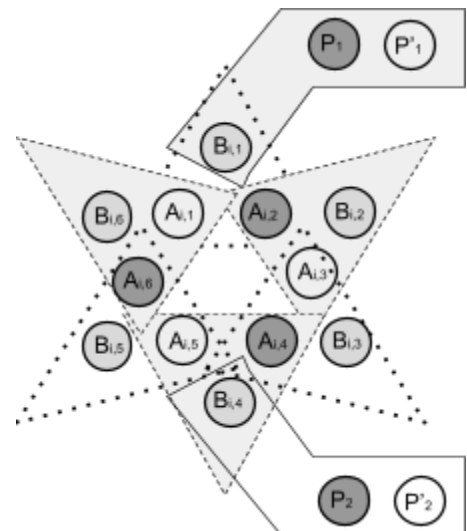
A los elementos puntas creados para la variable  $v_i$  las llamaremos  $B_{i,j}$  con  $0 < j \leq 2k$ . A los elementos core los

llamaremos  $A_{i,j}$  con  $0 < j \leq 2k$ . Los elementos  $A_{i,j}$  con valor  $j$  par los agregaremos al conjunto  $X$ . A los impares en el conjunto  $Y$  y a los elementos  $B_{i,j}$  los haremos parte del conjunto  $Z$ .

Crearemos triplas  $\{A_{i,j}, A_{i,j+1}, B_{i,j}\}$  para el conjunto  $C$ . Tendremos  $2k$  triplas en total por cada variable. La mitad de ellas con  $j$  par y el resto con  $j$  impar. Las llamaremos respectivamente triplas pares e impares.

Por cada cláusula " $j$ " crearemos 2 elementos  $p_j$  y  $p'_j$ . A los primeros los introduciremos en el conjunto  $X$  y a los segundos en el conjunto  $Y$ . Tendremos  $k$  elementos agregados en  $X$  e  $Y$ . Luego crearemos 3 triples por cada cláusula. Una por cada literal de la misma. Los primeros dos elementos de cada tripla serán los  $p_j$  y  $p'_j$ . El tercer elemento corresponderá a un elemento "punta"  $B_{a,b}$ . El literal corresponde a una variable negada o sin negar. Si corresponde a la variable  $v_i$  entonces la punta será una correspondiente a el gadget de esa variable ( $a=i$ ). Al ser la cláusula  $j$ , entonces reduce a las puntas  $b=2j$  o  $b=2j-1$ . Si la variable está sin negar entonces será el primer valor de  $b$ , y si está negada, el segundo.

En el ejemplo se muestran dos triplas por cláusula. La primera  $(P_1, P'_1, B_{i,1})$  corresponde a la correspondiente a la utilización de la variable  $\neg v_i$  (negada) en la primera cláusula de la fórmula. La segunda  $(P_2, P'_2, B_{i,4})$  corresponde a la utilización de la misma variable anterior, esta vez sin negar, en la segunda cláusula.



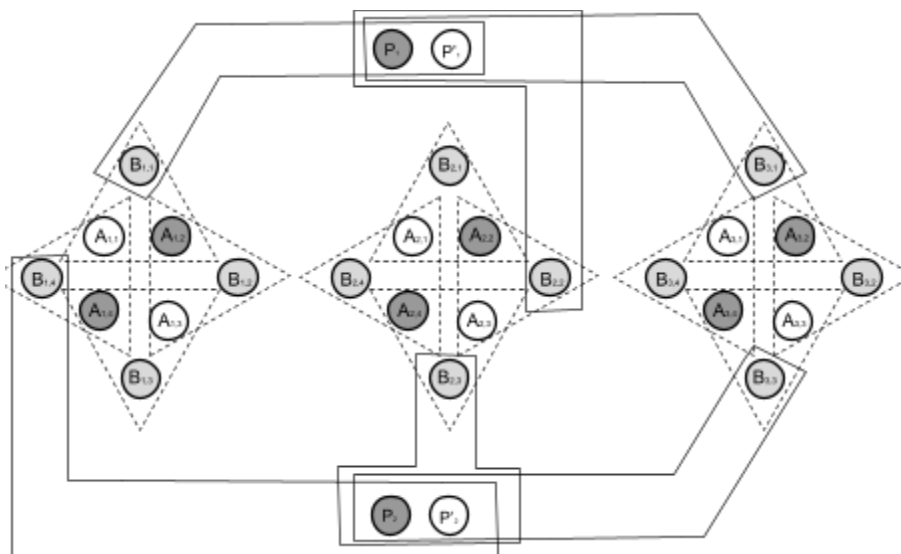
La idea detrás de esta transformación es que la selección de las diferentes triplas para cubrir todos los elementos están relacionados a la posibilidad de satisfacer la función booleana. A la hora de satisfacer una cláusula, al menos una de sus variables tiene que tener un literal en true. La satisfacción de la cláusula  $j$  está representada en nuestra transformación en la inclusión de una tripla en el matching que contenga a los elementos de  $X$  e  $Y$ :  $P_j, P'_j$ . Esto solo se puede lograr emparejándose con alguna de las puntas de los gadgets variables que contiene. Si la punta es impar entonces las puntas pares de esa gadget variable ya no podrán utilizarse en las triplas de otras cláusulas (y activarlas). Esto se puede ver en el ejemplo. Si  $(P_1, P'_1, B_{i,1})$  es una tripla seleccionada. Entonces  $(A_{i,1}, A_{i,1}, B_{i,1})$  ya no puede seleccionarse. Pero entonces la única forma de tener a  $A_{i,2}$  en algún

emparejamiento es seleccionar en el matching a  $(A_{i,2}, A_{i,3}, B_{i,2})$ . Ahora pasa lo mismo con  $A_{i,4}$  y eso provoca un encadenamiento de elección de triplas para lograr el matching. Esto en nuestro problema original de 3Sat significa que no podemos seleccionar una variable con dos valores diferentes. Si podemos utilizar otras puntas impares.

En el planteo hasta ahora hay un problema. Tenemos  $k$  cláusulas y  $2kn$  puntas. Al momento de cubrir las cláusulas utilizamos  $k$  puntas. Además  $kn$  estarán cubiertas por triplas del mismo gadget. Nos quedan  $(n-1)k$  puntas sin cubrir. Debemos entonces agregar un nuevo gadget que llamaremos "cleanup gadget" para resolverlo.

Crearemos en total  $(n-1)k$  "cleanup gadget". En cada uno de estos agregaremos 2 elementos. El primero  $Q_j$  lo incluiremos en el conjunto  $X$ . El segundo  $Q'_j$  en el conjunto  $Y$ . Luego crearemos una tripla por cada punta en los gadget variables que incluirá a  $Q_j$ ,  $Q'_j$  y esta. Tendremos entonces  $(n-1)k * 2kn$  triplas adicionales en  $C$ . Fácilmente entonces podemos cubrir con ellos las  $(n-1)k$  puntas restantes. De esta forma, si existe una manera de realizar un matching de todos los elementos, entonces es posible satisfacer la expresión. De lo contrario, no es posible.

Dado la reducción que se puede realizar en tiempo polinomial, podemos afirmar que el problema pertenece a NP-H. Dado que también corresponde a NP, ya podemos afirmar que el problema pertenece a NP-C.

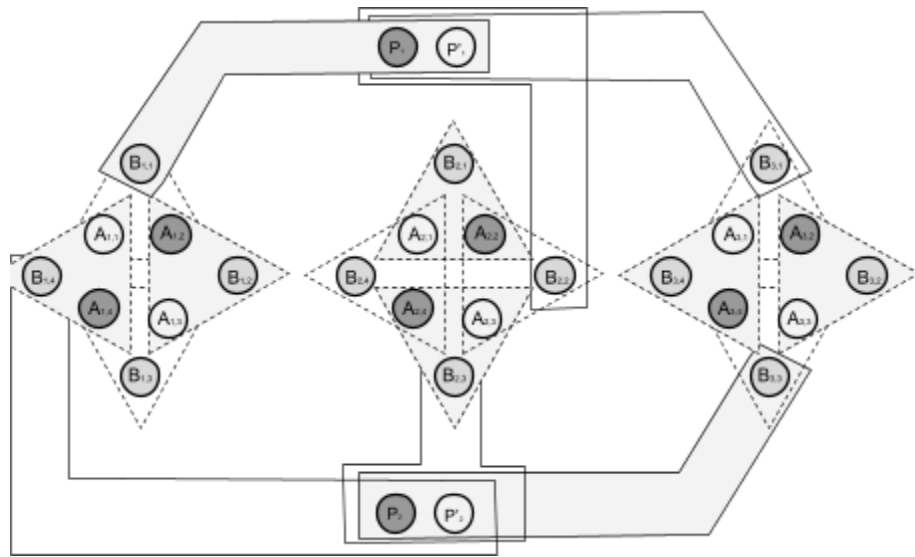


Supongamos que nuestra instancia de 3SAT consiste en  $\phi = (\neg v_1 \vee v_2 \vee \neg v_3) \wedge (v_1 \vee \neg v_2 \vee \neg v_3)$ . Con  $n=3$  y  $k=2$ . Para la reducción crearemos 3 gadgets de variables (por las 3 variables). Tendremos 12 elementos punta. Los incluiremos en  $Z$ . 12 elementos núcleo. A los

6 con  $j$  par los incluiremos en  $X$  y el resto en  $Y$ . Creamos 2 pares de elementos  $P_j$  y  $P'_j$  para las

cláusulas. De igual manera a los primeros de cada par los incluiremos en  $X$  y al resto en  $Y$ . Además creamos 4 pares de elementos  $Q_j$  y  $Q'_j$  para los cleanup gadgets. También dividiéndolos entre  $X$  e  $Y$ . Tendremos entonces  $X = \{A_{1,2}, A_{1,4}, A_{2,2}, A_{2,4}, A_{3,2}, A_{3,4}, P_1, P_2, Q_1, Q_2, Q_3, Q_4\}$ ,  $Y = \{A_{1,1}, A_{1,3}, A_{2,1}, A_{2,3}, A_{3,1}, A_{3,3}, P'_1, P'_2, Q'_1, Q'_2, Q'_3, Q'_4\}$ ,  $Z = \{B_{1,1}, B_{1,2}, B_{1,3}, B_{1,4}, B_{2,1}, B_{2,2}, B_{2,3}, B_{2,4}, B_{3,1}, B_{3,2}, B_{3,3}, B_{3,4}\}$ . Tenemos entonces en cada conjunto  $2nk=12$  elementos. Se deben crear las triplas. En la imagen se muestran todas las triplas que se deben crear, excepto las de los cleanup gadget. 12 triplas de los gadgets de variables y 6 triplas por las cláusulas. A estas se le deben sumar las 48  $(12*4)$  triplas de los cleanups gadgets.

En la siguiente imagen se muestra un posible matching. A las 4 puntas no cubiertas se las emparejará con los elementos de los cleanup gadget. Existen otras formas de colorear, sin embargo alcanza con una para que la expresión 3sat se pueda satisfacer.



### 3. Subset Sum

En este problema trabajaremos con una dimensión diferente con respecto a los que estábamos clasificando anteriormente. Lidiamos hasta el momento con grafos o solapamientos de conjuntos. En este caso si bien tendremos elementos de un conjunto que tendrán un valor numérico. Puede representar un peso, volumen, precio u otra entidad que caracteriza a cada uno de estos. Deberemos seleccionar un subconjunto de tal forma que la suma de los valores de estos sea igual a un valor dado.

El problema conocido como subset sum ya lo presentamos cuando trabajamos en la resolución de problemas mediante la técnica de programación dinámica. Está estrechamente emparentado con el problema de la mochila. En esa ocasión obtuvimos una

---

solución pseudopolinomial para resolverlo. Veremos a continuación que no existe una solución polinomial para resolver este problema (y tampoco el de la mochila) a menos que se demuestre que P es igual a NP. Demostrarnos que este problema pertenece al conjunto de problemas NP-C

### Subset Sum (SSS)

Dado un conjunto C de "n" números naturales queremos determinar si existe un subconjunto de ellos cuya suma sea exáctamente W.

Contamos con un catálogo de 10 productos con los siguientes valores: 7,3,19,23,50,55,40,4,30,12. Queremos saber si podemos seleccionar un subconjunto que sume 52. En este caso la respuesta es afirmativa. Lo podemos lograr sumando  $7+3+30+12$ . También hay otras alternativas como  $3+19+30$  o  $40+12$ . Sin embargo es imposible con estos sumar el valor 25 (al ser una instancia pequeña lo podemos probar mediante fuerza bruta).

Para probar que este problema es NP-C primero presentaremos un algoritmo certificador polinomial. La evidencia será un subconjunto de números que proponemos como posible solución. Deberemos probar que:

- Todos los números pertenecen al conjunto "C"
- Cada uno de los números no es utilizado más de una vez
- La suma de los mismos da como resultado W.

Cada una de estas operaciones la podemos realizar en tiempo polinomial. Por lo tanto podemos afirmar que es un problema perteneciente a NP.

Ahora debemos probar que pertenece a NP-H. Tomaremos otro problema que ya demostramos NP-C (y por lo tanto NP-H) y lo reduciremos a este. En varias de las últimas demostraciones hemos utilizado 3-SAT. También podríamos realizarlo. Pero para probar un camino diferente, utilizaremos 3DM para la reducción.

Partimos de una instancia de 3DM con tres conjuntos X,Y,Z de "n" elementos cada uno. Además un conjunto T de triplas. Lo transformaremos algorítmicamente y en tiempo polinomial en una instancia de Subset Sum. En esa transformación llegaremos a un conjunto "C" de números enteros y un valor W. Si es posible sumar mediante un

---

subconjunto de  $C$  el valor  $W$ , entonces afirmamos que es posible encontrar una cobertura de los conjuntos  $X, Y$  y  $Z$ . De lo contrario, no será posible.

Sabemos que cada conjunto  $X, Y, Z$  tiene " $n$ " elementos. Podemos tomar los elementos de  $X$  e identificarlos arbitrariamente como  $x_1, x_2, \dots, x_n$ . (cualquier orden es válido, pero debemos siempre mantener el mismo). Si tomamos un vector binario de  $n$  posiciones, podemos identificar al elemento  $x_i$  como aquel que tiene todos sus elementos en cero excepto el de posición  $i$ . De igual manera podemos representar a los elementos de  $Y$  e  $Z$ .

Para representar a cada una de las triplas del conjunto  $C$  podemos concatenar los vectores de los elementos de  $X, Y$  y  $Z$  que lo conforman. Por lo que tendremos un vector de longitud  $3n$  bits con exactamente 3 bits en "1" y el resto en "0".

*Sea la instancia de 3DM con  $X = \{x_1, x_2, x_3\}$ ,  $Y = \{y_1, y_2, y_3\}$ ,  $Z = \{z_1, z_2, z_3\}$  y  $T = \{(x_1, y_1, z_1), (x_2, y_2, z_3), (x_3, y_3, z_1), (x_1, y_1, z_2), (x_3, y_3, z_2), (x_1, y_3, z_2), (x_3, y_3, z_3)\}$ . Tenemos  $n=3$ . Podemos representar los elementos de  $X$  como 100, 010 y 001 respectivamente. Realizamos lo mismo con  $Y$  y  $Z$ . La representación de las triplas como concatenación de sus elementos corresponden a 100100100, 010010001, 001001100, 001001010, 001001010, 100001010 y 001001001 respectivamente. Son vectores de  $3*n=9$  posiciones.*

Recordamos que buscamos un subconjunto de  $C$  que sea un matching perfecto (utilizando la misma nomenclatura que en nuestro problema Gale-Shapley: todos los elementos están emparejados ni más ni menos que una vez). Si tenemos en cuenta la representación de las triplas en  $C$ , nuestro subconjunto debe ser " $n$ " vectores. Deben cumplir con ciertos requisitos para ser un matching perfecto. Cada posición " $i$ " de los vectores puede estar una y solo una vez con valor "1". De lo contrario implicaría que un mismo elemento fue seleccionado más de una vez o que no lo fue.

*En el ejemplo anterior podemos seleccionar el subconjunto de " $C$ " de los vectores 100100100, 010010001 y 001001010. Estos corresponden a los elementos  $\{(x_1, y_1, z_1), (x_2, y_2, z_3) \text{ y } (x_3, y_3, z_2)\}$ . Se puede ver que cada posición de los vectores aparecen una y solo una vez (ni más ni menos) con el valor "1". Por lo que corresponden a una cobertura perfecta y solución a la instancia del problema. Por otro lado, podríamos seleccionar a los vectores 001001010, 100001010 y 001001001. Pero estos no corresponden a una solución válida. La tercera posición del primer y*

---

tercer vector tienen valor "1". Visualizando las triplas originales  $\{(x_3, y_3, z_2), (x_1, y_3, z_2), (x_3, y_3, z_3)\}$  equivale a utilizar más de una vez al elemento  $X_3$ .

Hasta el momento hemos encontrado una manera diferente de representar los elementos y las triplas. Debemos transformar a estos en números de un conjunto y definir un valor  $W$  para llegar a una instancia de Subset Sum.

Si tomamos al vector lo podemos imaginar como la representación binaria de un número. De esa forma cada elemento es representado por este. Además si observamos la suma de todos los vectores que conforman una solución válida de una instancia veremos que debe ser un vector de la misma longitud que las triplas con todas las posiciones con valores en "1". Eso también representa un valor numérico que podríamos tomar como  $W$ . Con esto logramos realizar una transformación entre instancias. Por cada tupla creamos un número

calculado con su vector de representación como:  $\sum_{i=0}^{3n-1} v_i * 2^i$  y para  $W$  utilizamos  $\sum_{i=0}^{3n-1} 1 * 2^i$ .

Sin embargo, tenemos un problema grave con este planteo. Estamos viendo al vector como una representación binaria. Las sumas pueden generar desbordamientos (overflow) en las diferentes posiciones del vector. Si seleccionó dos o más números cuya representación vectorial tienen en la misma posición un "1" se genera en la suma un desborde que impacta en el valor de las posiciones siguientes. Para evitar eso se puede representar a cada elemento no con solo una posición, sino con más. ¿Cuántos más? Las necesarias para que sea imposible generar un desbordamiento que llegue a la representación del siguiente elemento. En el caso extremo todas las triplas pueden tener un mismo elemento. Si contamos con " $m$ " triplas, entonces necesitaremos que la representación de cada elemento en el vector tenga  $\lceil \log_2(m + 1) \rceil^{techo}$  posiciones.

Otra forma de resolverlo, es cambiar a una representación en otra base. Si el vector representa un número en base  $b = m + 1$ . Entonces aun teniendo todas las triplas el mismo elemento, la suma no llegará al próximo elemento. Tomaremos esta última representación. Por lo tanto calculamos cada número basándonos en su vector como

$\sum_{i=0}^{3n-1} v_i * b^i$  y para  $W$  utilizamos  $\sum_{i=0}^{3n-1} 1 * b^i$ . El resultado corresponde a los números de la

instancia de subset Sum y al  $W$  a conseguir con el subconjunto.

---

Supongamos que los números que seleccionamos en el ejemplo anterior corresponden a los representados por los vectores 010010001, 001001100 y 001001010. Si los sumamos  $010010001+001001100+001001010=100100111$  vemos que por más que no seleccionamos  $x_1$  (la primera posición del vector), la suma nos estaría informando que si lo realizamos. Se debe tener en cuenta que en el problema de 3DM la única manera de lograr una cobertura es seleccionar “n” triplas. Sin embargo en el problema de Subset Sum podemos elegir los números que necesitemos para llegar a W. Por lo tanto en algunas instancias seleccionando más de “n” números se podría llegar a la respuesta “true”. Sin embargo esta afirmación no sería válida para la instancia de 3DM original. No hay una correlación entre la solución de las instancias.

Para evitar el problema utilizaremos  $b = 7 + 1 = 8$  como base en la conversión de nuestro vector en el número. Tomamos  $(x_1, y_1, z_1)$  representado como 100100100 y lo convertimos en

un número como  $\sum_{i=0}^{3n-1} v_i * 8^i = 8^0 + 8^3 + 8^6 = 1 + 512 + 262144 = 262657$ . Realizando la

misma operación para el resto de las triplas tendremos respectivamente 16781320, 294976, 2097665, 2129984, 2129921 y 16810048. Por último W lo podemos calcular como

$$\sum_{i=0}^{3n-1} 1 * 8^i = 19173961.$$

Las triplas  $\{(x_1, y_1, z_1), (x_2, y_2, z_3) \text{ y } (x_3, y_3, z_2)\}$  que en el problema original corresponden a una solución, tienen asociados los números 262657, 16781320 y 2129984. Si los sumamos llegamos al número 19173961. Este número es W y corresponde a la solución de la instancia. Por lo tanto, si existe una solución en la instancia de Subset Sum, entonces existe en la de 3DM, y de lo contrario no.

Habiendo demostrado mediante la correspondiente reducción polinomial la pertenencia a NP-H y previamente su pertenencia a NP, ya quedó demostrado que el problema es NP-C.

## 4. Ciclos hamiltonianos

Sir William Rowan Hamilton fue un matemático Irlandés que nació a principios del siglo XIX. Entre sus muchos aportes, tal vez uno que no se sobresale, se encuentra la creación de un juego que denominó “juego icosiano” (icosian game). En este se presenta un dodecaedro regular en el que se solicita conformar un polígono utilizando las aristas de la figura y

pasando una vez (ni más ni menos) por cada de los vértices del mismo. En la imagen vemos una vista plana del dodecaedro donde se remarca una posible solución del juego. Podemos abstraer la figura mediante un grafo. Los vértices de la misma corresponden a los nodos y las aristas a los ejes. Buscamos un ciclo que inicia por un vértice, pasa por cada uno de los otros solo una vez y finaliza en el mismo que inició. Se conoce a esté como ciclo Hamiltoniano en honor al matemático.



Históricamente el mismo problema se estudió en diferentes contextos. Por ejemplo el conocido como tour del caballo (Knighth tour) corresponde a recorrer el tablero de ajedrez mediante la pieza caballo pasando por todas los casilleros comenzando y finalizando por el mismo, sin pasar dos veces por el mismo.

Estamos ya en condiciones de presentar el problema:

### Ciclo Hamiltoniano (CH)

Dado un grafo  $G=(V,E)$  queremos construir un ciclo que utilice todos los vértices una y solo una vez. Iniciando y finalizando por el mismo.

Para demostrar que este problema pertenece a la clase NP-H, iniciaremos con su certificador polinomial. Como evidencia utilizaremos una lista de nodos que conforman el ciclo. El algoritmo deberá verificar que:

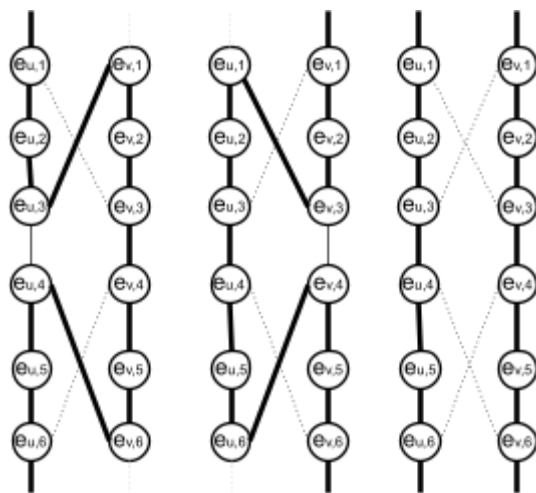
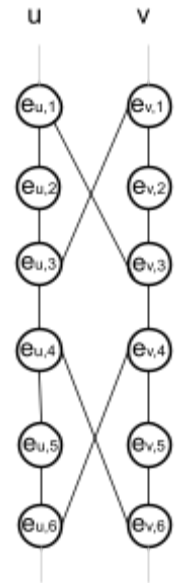
- Todos los nodos pertenecen al grafo
- No falta utilizar ninguno de los nodos
- No se utilizan ninguno de los nodos más de 1 vez (excepto el primero y el último)
- El primero y último nodo de la lista deben ser el mismo.
- Para cada nodo y su contiguo en la lista existe un eje en el grafo que los une.

Todos estas operaciones las podemos realizar en tiempo polinomial. Por lo tanto podemos afirmar que el problema pertenece a NP.

La demostración de su pertenencia a NP-H la realizaremos reduciendo el problema de cobertura de vértices (vertex Cover). Presentaremos un proceso algorítmico para reducir cualquier instancia de vertex cover en el problema del ciclo hamiltoniano. Recordemos antes el enunciado de este problema. Contamos con un grafo  $G=(V,E)$  y queremos encontrar un subconjunto de nodos igual o menor a  $k$  tal que todos los ejes del grafo tengan en sus extremos al menos uno de los nodos del subconjunto.

Para la reducción partimos de un grafo  $G=(V,E)$  y un valor  $k$ . Lo transformaremos en un grafo  $G'=(V',E')$  en el que buscaremos el ciclo hamiltoniano. La transformación busca que existe un ciclo hamiltoniano si y solo se puede hacer la cobertura de vértices.

Crearemos un "gadget de eje" por cada eje  $e=(u,v) \in E$ . Este gadget estará conformado por 12 nodos y 14 ejes que agregaremos al grafo  $G'=(V',E')$ . Cada gadget estará dividido en 2 grupos de 6 nodos alineados entre sí. Los primeros de ellos corresponden al nodo "u" y los segundos al nodo "v". Están interconectados entre sí por ejes, como se muestra en la imagen adjunta. La comunicación del gadget al exterior se da únicamente a través de los nodos  $e_{u,1}$ ,  $e_{u,6}$ ,  $e_{v,1}$  y  $e_{v,6}$ . Llamaremos a estos últimos "nodos



conectores". Para que exista un ciclo hamiltoniano todos los nodos deben ser visitados y esto se puede hacer de 3 formas diferentes. Las primeras dos mediante una visita al gadget: el camino  $(e_{u,1}, e_{u,2}, e_{u,3}, e_{v,1}, e_{v,2}, e_{v,3}, e_{v,4}, e_{v,5}, e_{v,6}, e_{u,4}, e_{u,5}, e_{u,6})$  y el camino  $(e_{v,1}, e_{v,2}, e_{v,3}, e_{u,1}, e_{u,2}, e_{u,3}, e_{u,4}, e_{u,5}, e_{u,6}, e_{v,4}, e_{v,5}, e_{v,6})$ . La última mediante dos visitas  $(e_{u,1}, e_{u,2}, e_{u,3}, e_{u,4}, e_{u,5}, e_{u,6})$  y  $(e_{v,1}, e_{v,2}, e_{v,3}, e_{v,4}, e_{v,5}, e_{v,6})$ . El sentido de los recorridos puede ser el mostrado o el inverso para conformar el ciclo hamiltoniano.

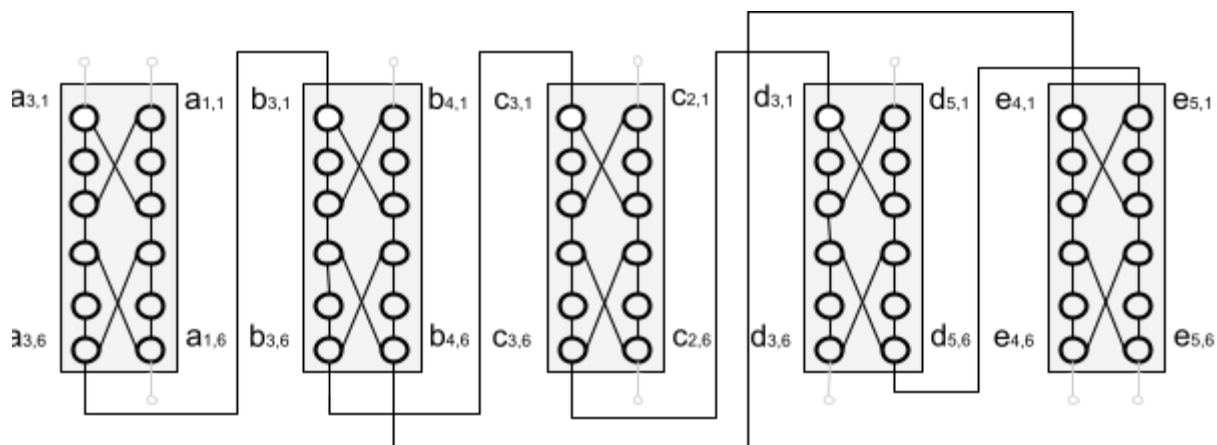
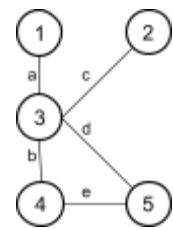
Cada nodo  $u \in V$  aparecerá en tantos gadgets como el grado del mismo (el número de ejes adyacentes).

Agregaremos ejes entre los nodos conectores de los gadgets para generar caminos que recorran todos los nodos  $e_{u,x}$  de  $V'$  con  $1 \leq x \leq 6$ . El orden de la conexión no es importante y puede ser arbitrario. Pero es importante definirlo a priori. Si el grado del vértice  $u$  es " $g$ ", tendremos  $g$  gadgets generados con " $u$ " en algunos de los dos lados de los nodos. recorriendo según el orden fijado los llamaremos  $e^1, e^2, \dots, e^{(g)}$ . Para crear el camino requiere agregar los ejes  $(e^1_{u,6}, e^2_{u,1}), (e^2_{u,6}, e^3_{u,1}), \dots, (e^{(g-1)}_{u,6}, e^{(g)}_{u,1})$ .

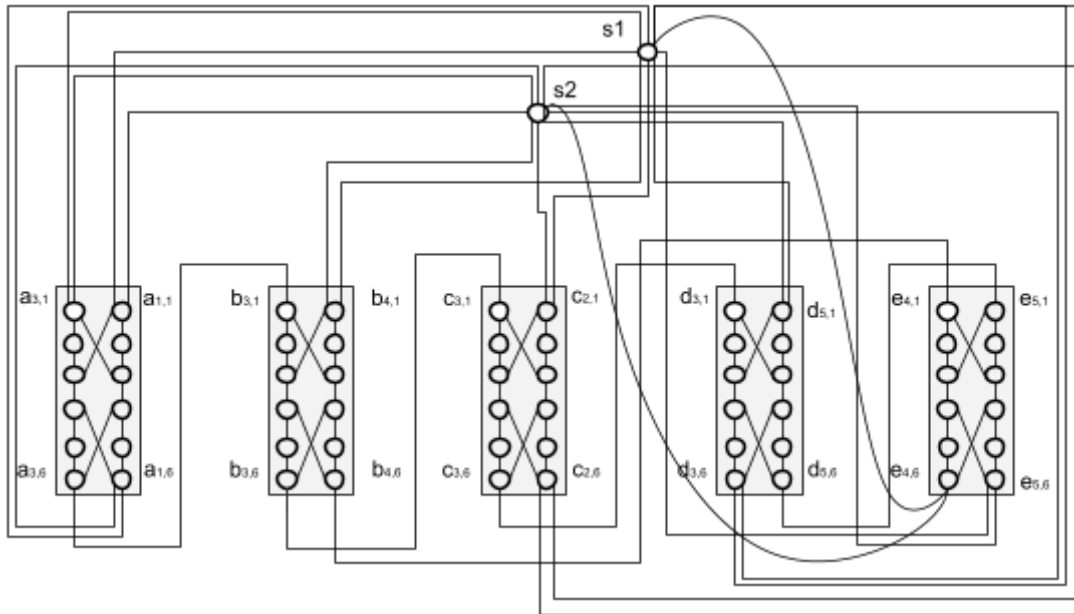
Por último agregaremos " $k$ " nodos  $s_1, s_2, \dots, s_k$  que llamaremos "nodos de selección". Estos nodos se conectarán a los nodos conectores de cada uno de los gadgets que no están conectados por otros ejes externos. El grafo resultante corresponde a la instancia del problema de ciclo hamiltoniano.

De ser posible construir un ciclo hamiltoniano los nodos selección serán cruciales para informar cuales son los nodos del grafo  $G$  que se deben seleccionar para realizar la cobertura de vértices. Estos nodos serán aquellos vinculados a los nodos conectores dentro del ciclo.

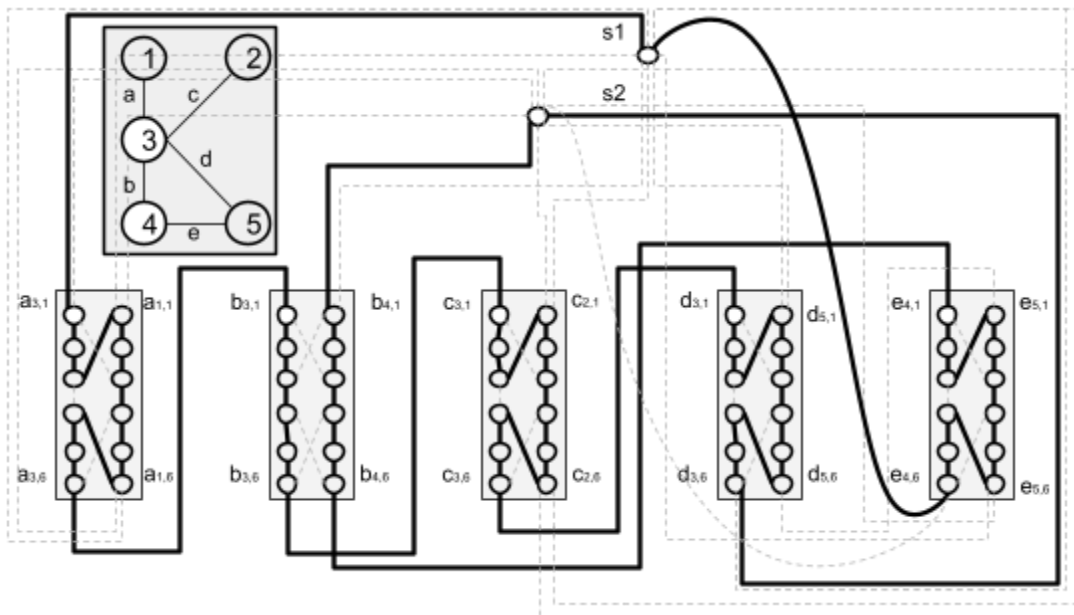
Dado el grafo  $G=(V,E)$  de 5 nodos y 5 ejes presentado en la imagen queremos saber si es posible realizar una cobertura de vértices con  $k=2$  o menos nodos. Para la reducción al problema de ciclo hamiltoniano tendremos que crear 5 gadgets de eje (por los 5 ejes) y 2 nodos de selección (por  $k$ ). Iremos paso a paso. Tendremos para comenzar en el grafo  $G'=(V',E')$  que incluir  $5 \cdot 12$  nodos y  $5 \cdot 14$  ejes por los gadgets. Luego debemos agregar los ejes para formar los caminos que vinculan a todos los nodos de  $V$  dentro de los gadgets. Tendremos 3 ejes por el nodo 3 (se encuentra en 4 gadgets al ser grado 4), 1 eje por el nodo 4 (grado 2), 1 por el nodo 5. Los nodos 1 y 2 no aportaran ejes adicionales al tener grado 1.



Resta agregar los  $k=2$  nodos de selección. Debemos conectarlos con ejes a los nodos externos de los gadgets que conexión que no tienen conexiones con los otros gadgets. Serán, para nuestro ejemplo, 10 ejes por cada nodo de selección. En total  $2 \cdot 10$  ejes adicionales. Vemos en la imagen como queda el grafo  $G'$  final para la instancia reducida. Contiene 62 nodos y 95 ejes.



Si encontramos un camino hamiltoniano, será posible la cobertura de vértices con dos nodos. En la siguiente imagen se muestra que esto es posible.



---

*Si observamos atentamente podemos ver que aquellos nodos con subíndice 1, por los que llegan los ejes desde los nodos  $s_1$  y  $s_2$ , están relacionados con los nodos 3 y 4 de la instancia del problema original:  $a3,1$  y  $b4,1$ . De esa manera podríamos incluso determinar el subconjunto de nodos solución del problema original. No es la única solución posible. Queda para el lector encontrar un camino hamiltoniano alternativo que tenga como resultado vinculado en el vertex cover a los nodos 3 y 5.*

Pero, ¿cómo sabemos que esta transformación es siempre válida?. Profundizaremos en el análisis de la misma para determinarlo. Supongamos que conocemos a priori el subconjunto  $U$  de vértices  $u_1, u_2, \dots, u_k$  de  $G$  que conforman el vertex cover (si el vertex cover que conocemos del grafo tiene menos que  $k$  vértices podemos agregar arbitrariamente nodos hasta llegar a este valor sin que deje de serlo). Para construir el ciclo hamiltoniano en su grafo transformado simplemente debemos: Agregar los ejes  $(s_j, e^1_{u_j,1})$  con  $1 \leq j \leq k$ .

Además todos los ejes que conectan que conforman los caminos entre gadgets  $(e^1_{u_j,6}, e^2_{u_j,1})$ ,  $(e^2_{u_j,6}, e^3_{u_j,1}), \dots, (e^{(g-1)}_{u_j,6}, e^{(g)}_{u_j,1})$  con  $1 \leq j \leq k$ . La forma de unir los nodos internos de cada gadget está determinada por si los nodos de  $V$  que contiene se encuentran en  $U$  o no. Si ambos están en el subconjunto la forma de unirlos es mediante la configuración de los dos caminos. Si solo uno, con un solo camino determinado por los ejes entrantes. ¿Puede un gadget no tener ejes entrantes? La respuesta es que no. Si sabemos que es un vertex cover. El gadget es la representación de un eje. Que ninguno de sus nodos extremos tenga un eje seleccionado implica que no está cubierto. Lo que sería absurdo.

Por último, para cerrar el ciclo se deben agregar los ejes que regresan a los nodos selección  $(e^{(g)}_{u_j,6}, s_{j+1})$  con  $1 \leq j < k$  y  $(e^{(g)}_{u_k,6}, s_1)$ .

Si comenzamos por el nodo  $s_1$ , vemos que ingresa al  $u_1$ , recorremos todos los otros gadgets que tiene  $u_1$  hasta ir a  $s_2$ . Desde allí podemos visitar todos los gadgets con  $u_2$  e ingresar a  $s_3$ . Seguir con el procedimiento hasta volver a  $s_1$ . Estamos por lo tanto en presencia de nuestro ciclo hamiltoniano.

Si partimos del ciclo hamiltoniano, afirmamos los ejes  $(s_j, e_{u_j,1}^1)$  con  $1 \leq j \leq k$  nos permiten obtener la cobertura de vértices formados por los  $u_j$  de estos. Para demostrarlo primero partiremos el ciclo hamiltoniano en caminos que van de  $s_j$  a  $s_{j+1}$  con  $1 \leq j < k$  y finalmente de  $s_k$  a  $s_1$ . Podemos observar que estos tienen que ser  $k$  segmentos y que tienen que entrar inicialmente algún gadget, puede pasar por otros para luego ir al nodo selector. Por cada segmento  $j$  nos quedaremos con concentrarnos con el eje ingresante  $(s_j, e_{u_j,1}^1)$ . Este representa al nodo  $u_j$  del vertex cover. El camino va ingresando por otros gadgets que representan ejes con los que  $u_j$  está en contacto hasta llegar con el siguiente nodo selector. Representa todos los ejes que cubre el nodo. Como es un camino hamiltoniano todos los gadgets son visitados. Y al obtener todos los  $u_j$  de cada segmento tenemos los nodos del vertex cover.

## 5. Ciclo hamiltoniano dirigido y viajante de comercio.

En esta sección trabajaremos con variantes de problemas anteriores. Mostraremos que muchas veces ciertos problemas tienen una reducción muy sencilla si partimos de problemas en los que tuvimos que hacer un esfuerzo mental para demostrar la reducción.

En primer lugar nos preguntamos, en qué cambiaría si al buscar un ciclo hamiltoniano lo hacemos sobre un grafo dirigido. Es decir:

| Ciclo Hamiltoniano dirigido(CHD)                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dado un grafo $G=(V,E)$ con ejes dirigidos queremos construir un ciclo que utilice todos los vértices una y solo una vez. Iniciando y finalizando por el mismo. |

Probar que el problema pertenece a la clase NP es similar al problema con el grafo no dirigido. Solo debemos agregar ahora que el eje utilizado en el ciclo tenga la dirección utilizada en el grafo.

Para probar que pertenece a NP-H reduciremos el problema de ciclo hamiltoniano a este. La reducción es muy sencilla. Partimos de un grafo  $G=(V,E)$ . Primero crearemos un nuevo grafo  $G'=(V',E')$  con los mismos nodos  $V'=V$ . Luego crearemos por cada eje  $e=(u,v) \in E$  dos

---

pares de ejes dirigidos  $e'_1=(u,v)$  y  $e'_2=(v,u)$  y los incluiremos en  $E'$ . En tiempo polinomial construimos una instancia de CHD. Si existe un ciclo hamiltoniano dirigido utiliza alguno de cada uno de los dos ejes.

Pasemos al problema del viajante de comercio. En su versión de decisión lo podemos enunciar así:

| Problema del viajante de comercio |
|-----------------------------------|
|-----------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contamos con un conjunto de “n” ciudades que visitar. Existen caminos que unen pares de ciudades. Cada camino inicia en una ciudad “x” y finaliza en la ciudad “y” tiene asociado un costo de tránsito de $w_{x,y}$ . Partiendo desde una ciudad inicial y finalizando en la misma se quiere saber si se puede construir un circuito que visite cada ciudad una y solo una vez con un costo total menor o igual a W. |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Un error común a la hora de trabajar con este problema es plantear que la instancia corresponde a un grafo pesado dirigido. Sin embargo, si bien claramente en una abstracción lo podemos ver de esa manera, debemos plantearlo en términos y variables del problema: ciudades y rutas.

Para probar que el problema pertenece a NP debemos presentar un certificador polinomial. La evidencia corresponderá a un listado de ciudades ordenadas que corresponde a una posible solución. Debemos verificar que:

- La primera y última ciudad es la misma
- Todas las ciudades aparecen 1 y solo 1 vez (excepto la ciudad de inicio que aparece también al final)
- Existe un camino para cada par de ciudades del recorrido (si los caminos tienen sentido que lo respete)
- Que la suma de los costos de los caminos sea igual o menor a W.

Sin problema podemos ver que con la estructura correcta de la instancia y el certificador lo podemos realizar en tiempo polinomial.

Para demostrar que el problema es NP-H buscamos un problema del cual partir. Usaremos el ciclo hamiltoniano (dirigido en caso de tener los caminos sentido). Partimos del grafo

---

$G=(V,E)$  y la pregunta si es posible generar un circuito hamiltoniano. Para la reducción crearemos por cada nodo de  $V$  una ciudad. Luego por cada eje de  $E$  creamos un camino. El peso del camino lo definiremos en 1. Finalmente el valor  $W=|V|-1$ . De esa forma si existe un recorrido por todas las ciudades con costo  $W$ , entonces existe un ciclo hamiltoniano. Se ve claramente que para lograr el costo de  $W$  se tiene que pasar por  $|V|-1$  caminos de costo 1. Si no existe el circuito entonces no hay ciclo hamiltoniano. La reducción es en tiempo polinomial a la instancia del problema del ciclo hamiltoniano.

Con esto demostramos que el problema pertenece a NP-H y habiendo demostrado que pertenece a NP, corresponde a un problema NP-C.

## 6. Problema de la mochila

Para finalizar abordaremos el problema de la mochila. En su versión de decisión:

| Problema de la Mochila                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contamos con una mochila con una capacidad de $K$ kilos y queremos saber si podemos introducir dentro de ellas un subconjunto del conjunto $E$ de " $n$ " elementos con el objetivo de conseguir una ganancia de $V$ o más. Cada elemento $i$ tiene un peso de $k_i$ kilos y un valor de $v_i$ . Los elementos no pueden ser divididos y el peso total seleccionado no puede superar la capacidad de la mochila. |

Podemos construir un certificador polinomial que, tomando como evidencia un subconjunto de elementos, verifique que:

- Todos los elementos pertenecen a la instancia del problema
- Cada elemento se utiliza solo una vez
- La suma de los pesos de los elementos sea menor o igual a  $K$
- La suma de los valores de los elementos sea mayor o igual a  $V$ .

Por lo tanto demostramos su pertenencia a la clase NP.

Para demostrar que pertenece a NP-H se puede encontrar una similitud con el problema de subset sum. Por lo que utilizaremos este problema como punto de partida. Recordemos

---

que en este problema contamos con un conjunto de “n” números y queríamos saber si podemos seleccionar un subconjunto con suma igual a S.

Tomamos los “n” números y los convertiremos en igual cantidad de elementos. El peso y el valor de cada uno de ellos será el del número. La capacidad de la mochila será  $W=S$ . Finalmente el valor a conseguir también será  $V=S$ .

Vemos que cada subconjunto de elementos que podemos poner en la mochila tendrán exactamente el mismo peso y valor. Por lo tanto el máximo valor que se puede lograr sin superar la capacidad de la mochila es S. Cualquier valor menor es invalido por restricción de valor del problema. Es decir que si el problema de la mochila tiene solución, tendrá como peso y ganancia V. El subconjunto de elementos de esta solución representa al conjunto de números que en conjunto suma S. Si no hay resultado en la instancia de la mochila, tampoco la hay en subset sum,

Nuevamente la reducción es en tiempo polinomial. Con eso demostramos que el problema es NP-H. Como pertenece a NP y NP-H también pertenece a NP-C. Como queríamos demostrar.